Chapitre 0 — Avant tout | Partie B — Le temps | leçon 0.15 sur 153 | difficulté 1 / 10

0.15. Deux rythmes, un seul chronomètre

Le même programme en plus court : un seul compteur, et le modulo fait le reste.

1. Ce qu'il faut comprendre

C'est le programme du 0.14, en plus simple. L'écran est exactement le même : la lettre du bas change quatre fois pendant que celle du haut change une fois.

Un seul chronomètre : au lieu de deux compteurs, `images` compte de 0 à 59, puis repart à 0. Il fait donc un tour complet **chaque seconde**.

La lente change quand le chronomètre repasse par **0** : une fois par tour, donc une fois par seconde.

La rapide change quand `images % 15 == 0` : quand `images` est un **multiple de 15**, c'est-à-dire 0, 15, 30 ou 45. Le reste de la division par 15 ne vaut 0 que pour ces nombres : 30 % 15 = 0, mais 31 % 15 = 1. Cela fait **4 fois** par tour, donc 4 fois par seconde.

Le retour à A sans `if` : `(rapide + 1) % sizeof(ALPHABET)` ajoute 1, puis garde le reste de la division par 26. Tant qu'on est sous 26, le reste est le nombre lui-même (7 % 26 = 7) ; à 26, il retombe à **0** (26 % 26 = 0) : après Z, A.

Ce qu'on gagne : 3 variables au lieu de 4, deux `if` d'une ligne au lieu de deux blocs, et un programme plus léger. **Ce qu'on perd :** les deux rythmes sont liés au même chronomètre ; au 0.14, on pouvait choisir n'importe quelle durée pour chacun, sans rapport entre eux.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <reste>` → 0.8.4. #include <reste> — a % b
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <reste>  // a % b, sauf par 1, 2, 4, 8, 16… écrits en clair`	Demande à la console « reste » (ce que c'est : a % b, sauf par 1, 2, 4, 8, 16… écrits en clair). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « a % b, sauf par 1, 2, 4, 8, 16… écrits en clair ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`uint8_t lente = 0;  // QUELLE lettre montre la lente : 0 = A … 25 = Z`	Une variable : une boîte appelée lente, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « QUELLE lettre montre la lente : 0 = A … 25 = Z ».)
`uint8_t rapide = 0;  // QUELLE lettre montre la rapide : 0 = A … 25 = Z`	Une variable : une boîte appelée rapide, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « QUELLE lettre montre la rapide : 0 = A … 25 = Z ».)
`uint8_t images = 0;  // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde`	Une variable : une boîte appelée images, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // attend l'image suivante : 60 tours par seconde`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attend l'image suivante : 60 tours par seconde ».)
`// Le chronomètre avance d'une image, et repart à 0 après 59 :`	Un commentaire : une phrase pour celui qui lit (« Le chronomètre avance d'une image, et repart à 0 après 59 : »). La console l'ignore.
`// il fait un tour complet chaque seconde.`	Un commentaire : une phrase pour celui qui lit (« il fait un tour complet chaque seconde. »). La console l'ignore.
`images++;`	images augmente de 1.
`if (images == 60) images = 0;`	Une décision : si images est égal à 60 alors on fait « images = 0 ». Sinon, on passe.
`// La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).`	Un commentaire : une phrase pour celui qui lit (« La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde). »). La console l'ignore.
`//  images % 15 = le reste de images divisé par 15 ;`	Un commentaire : une phrase pour celui qui lit (« images % 15 = le reste de images divisé par 15 ; »). La console l'ignore.
`//  il vaut 0 seulement pour les multiples de 15.`	Un commentaire : une phrase pour celui qui lit (« il vaut 0 seulement pour les multiples de 15. »). La console l'ignore.
`// (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,`	Un commentaire : une phrase pour celui qui lit (« (rapide + 1) % sizeof(ALPHABET) : la lettre suivante, »). La console l'ignore.
`//  et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.`	Un commentaire : une phrase pour celui qui lit (« et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A. »). La console l'ignore.

`if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);`	Une décision : si images % 15 est égal à 0) rapide = (rapide + 1) % sizeof(ALPHABET alors on fait « ». Sinon, on passe.
`// La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).`	Un commentaire : une phrase pour celui qui lit (« La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde). »). La console l'ignore.
`if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);`	Une décision : si images est égal à 0) lente = (lente + 1) % sizeof(ALPHABET alors on fait « ». Sinon, on passe.
`poser(0, 0, ALPHABET[lente]);  // ligne 0 : la lente`	Appel de la fonction poser de la console, avec les arguments 0, 0, ALPHABET[lente]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « ligne 0 : la lente ».)
`poser(0, 2, ALPHABET[rapide]);  // ligne 2 : la rapide`	Appel de la fonction poser de la console, avec les arguments 0, 2, ALPHABET[rapide]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « ligne 2 : la rapide ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **950 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        58 o  ← la console, à l'allumage  — la mise en route de la console
  main()           94 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  Reste            10 o  ← main()  — le reste d'une division
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard,
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 2    24 o sur place, dans main()
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>   // a % b, sauf par 1, 2, 4, 8, 16… écrits en clair
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <poser>      // pose une tuile sur une case du fond
```

2. UNE VARIABLE — LENTE

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A … 25 = Z
```

3. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;   // QUELLE lettre montre la rapide : 0 = A … 25 = Z
```

4. UNE VARIABLE — IMAGES

```
uint8_t images = 0;   // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();                        // attend l'image suivante : 60 tours par seconde

    // Le chronomètre avance d'une image, et repart à 0 après 59 :
    // il fait un tour complet chaque seconde.
    images++;
    if (images == 60) images = 0;

    // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
    //   images % 15 = le reste de images divisé par 15 ;
    //   il vaut 0 seulement pour les multiples de 15.
    // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
    //   et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
    if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

    // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
    if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

    poser(0, 0, ALPHABET[lente]);    // ligne 0 : la lente
    poser(0, 2, ALPHABET[rapide]);   // ligne 2 : la rapide
  }
}
```

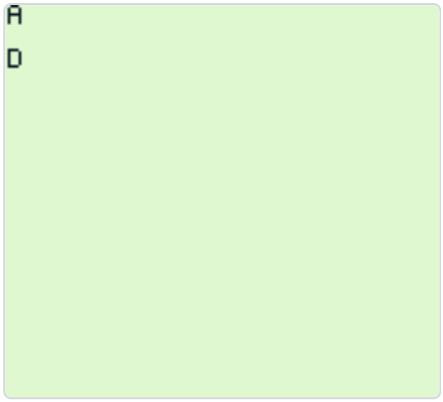
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

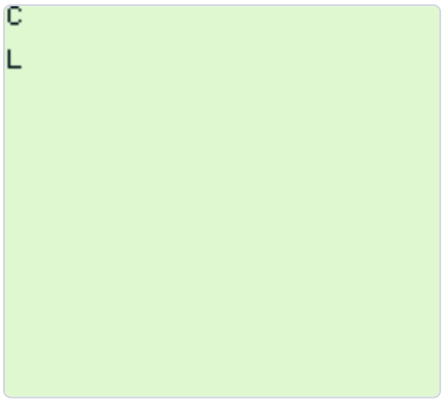
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Comme au 0.14 : la lettre du bas change quatre fois pendant que celle du haut change une fois.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la rapide change à chaque quart de seconde (8 fois en 2 s) (AB AC AD BE BF BG BH CI CJ)
- ✓ la lente change toutes les secondes (2 fois en 2 s)
- ✓ après Z, on revient bien à A (les lettres restent des lettres)

Ensuite : **0.15.1. Deux rythmes, un seul chronomètre — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++